



Deep Reinforcement Learning Control for Aerobatic Maneuvering of Agile Fixed-Wing Aircraft

Shanelle G. Clarke* and Inseok Hwang†
Purdue University, West Lafayette, IN 47906

Autonomous aerobatic flight has been a challenging control problem for many years. This is because aerobatic flight requires such highly precise control while operating on the extreme edges of the flight envelope which most controllers are not able to handle. For pilots, this precise control has been learnt through many years of flight experience. The research in this paper significantly shortens this learning time by extending the state of the art work in Deep Reinforcement Learning to the realm of flight control. This paper presents a Normalized Advantage Function controller that, unlike traditional controller architectures, emulates stick-to-surface control by utilizing a simple unified nonlinear control framework to command and coordinate full multi-actuator authority. Through trial-and-error simulated experiences, the controller is able to explore the full range of the nonlinear flight envelope and learn by itself, without human input, an aerobatic maneuver in little time on the order of hours. This controller utilizes the large multidimensional state and action spaces of the aircraft to optimize aerobatic performance and high degree autonomous flight skill. We are able to show through high-fidelity simulations that this controller is able to successfully learn and execute two different aerobatic maneuvers: Slow Roll and Knife Edge.

I. Nomenclature

$\pi(u x)$ or π	=	deterministic policy that governs the distribution over actions u given states x
$ \bullet $	=	absolute value of $\bullet$
A^T	=	transpose of the matrix A
$\exp(x)$	=	exponential of x
$\text{diag}(A)$	=	diagonal matrix of the square matrix A
$\text{tril}(A, -1)$	=	lower triangular matrix with diagonal zero elements of the matrix A
∇_θ	=	gradient of a function with respect to the variable θ
$\mathbb{E}_{u_t \sim \pi; x_t \sim \mathcal{P}}$	=	Expectation given that u is sampled from a distribution π and x_t is sampled from a distribution $\mathcal{P}$ at time t
$\mathbb{P}$	=	Probability
$\mathcal{P}$	=	Probability distribution
$\tanh$	=	hyperbolic tangent function
X	=	linear x -position in the body frame of the aircraft
Y	=	linear y -position in the body frame of the aircraft
Z	=	linear z -position in the body frame of the aircraft
v_X	=	x component of the linear velocity of the aircraft
v_Y	=	y component of the linear velocity of the aircraft
v_Z	=	z component of the linear velocity of the aircraft
ϕ	=	roll angle of the aircraft
θ	=	pitch angle of the aircraft
ψ	=	yaw angle of the aircraft
p	=	roll rate of the aircraft
q	=	pitch rate of the aircraft
r	=	yaw rate of the aircraft
Subscripts		
t	=	at time instance t

*Graduate Research Assistant, Department of Aeronautics and Astronautics, AIAA Student Member, Email: clarke24@purdue.edu

†Professor, Department of Aeronautics and Astronautics, AIAA Associate Fellow, Email: ihwang@purdue.edu

II. Introduction

ONE of the limiting factors in autonomous flight control for Fixed-Wing aircraft is the inability of traditional control methods to explore and utilize its full flight envelope. Traditional derivative-dependent control approaches make simplification assumptions that constrain the flight envelope as they are not equipped to handle the highly nonlinear and discontinuous aerodynamic behavior and coupling effects that occur on the edges of the flight regime [1]. In particular, there are few controllers that can effectively emulate aggressive and aerobatic flight maneuvers that operate on the boundaries of the flight envelope.

The idea of autonomous flight maneuvering for Fixed-Wing aircraft has been explored by a number of researchers. In 2005, Green et al [2] developed a hover configuration specific controller that can handle the autonomous transition from steady level flight to a hovering position. Since then, a number of researchers, [3–6], have iterated and optimized on autonomous hover flight control with a variety of traditional control approaches including PID control, Model Predictive Control (MPC) and Time Varying Linear Quadratic Regulator (TVLQR). While these approaches were able to successfully hover, they have failed to capture the aggressiveness of truly aerobatic flight. This is because their performance were circumscribed by either the linearization of the aircraft dynamics as in [3, 6] or the oversimplification in the approximate aerodynamics of the aircraft in the post stall regime as in [4, 5]. This simplification of the dynamics and/or aerodynamics is not able to capture the richness in flight behavior at the edges of the flight regime, which is due to the strong dominating nonlinearities in the separation of flow over lifting surfaces [1]. Extending beyond autonomous hovering and perching, both [7] and [8] executed an aggressive turn-around and Knife Edge maneuver of a RC Yak54 aircraft by optimizing over a nonlinear trajectory for motion planning. Formulated as an Optimal Control Problem (OCP), the controller architecture boasted a nonlinear motion planner that computes a feasible trajectory based on simplified nonlinear aircraft dynamics [7]. The desired feasible states from this optimized reference trajectory is segmented and input to a feedback PID controller with tuned gains. The most immediate and prominent challenge to using this method in real time is that the feasible trajectories are computed offline. In real time flight subject to disturbances and uncertainties in the plant, these precomputed trajectories may no longer be feasible for the aircraft. In fact, from [9], we can see that the actual flight path veers significantly from the desired computed path.

In an attempt to mitigate the issues faced in the above methods, Bulka et. al [10] designed and implemented a feedback PID controller that is able to handle the full nonlinear dynamics and aerodynamics as modeled in [11]. In contrast to [12], Bulka was able to experimentally implement his controller. While Ure, in [12], was able to design a sophisticated multimodal flight control architecture that was able to string maneuver primitives together to perform a sequence of maneuvers, he is unable to embed this formulation onboard an aircraft due to its large computational overhead. Another sophisticated control scheme was proposed by [13] which used a Lyapunov backstepping quaternion-based control technique. Both Sobolic and Bulka demonstrated the impressive capabilities of their control architectures, both in simulation and actual hardware implementation. While the current iteration of our proposed controller is not yet ready for hardware implementation, we show a less complex unified control scheme that is able to explore and exploit the rich behaviors of the nonlinearities on the edges of the flight envelope to demonstrate aerobatic learning. To do so, we incorporate current state-of-the-art machine learning techniques.

This paper capitalizes on the groundbreaking research and technologies developed in a subset of machine learning called Reinforcement Learning (RL). The objective of Reinforcement Learning is to maximize rewards through experiential learning. A survey of deep reinforcement learning techniques [14] has enumerated the successes of Model-Free RL approaches from Alpha-Go Games [15] to quadcopter stabilization control [16]. In fact, there has been notable, even if little, research on the use of RL techniques for rotorcraft control. In [17], a differential dynamic programming (DDP) approach was used to estimate and optimize over the learnt model dynamics to derive a control policy for aerobatic maneuvering of a helicopter. The model dynamics was learnt from flight data collected from expert pilots and fitted to a nonlinear second-order Ordinary Differential Equation (ODE). The control policy is based on an LQR feedback controller using the linear approximation to the learnt dynamics. In [18], Abbeel improved on this previous work by using expert pilot trajectory data to encode the maneuvers themselves, called Apprenticeship Learning. A generative model for each trajectory is created and used to calculate the reward signal via which the policy is improved. However, the policy is still modeled as a LQR controller that computes a control input at an operating point for the linearized flight dynamics.

For Fixed-Wing aircraft, there is little literature in the application of machine learning for flight control. To the best of our knowledge, in the area of aerobatic maneuvering, where we see the greatest applicability for groundbreaking research, there has been none. Given this lack, the work in this paper impresses its contribution by the following measures:

- The proposed controller is novel in that it applies state-of-the-art theories in machine learning towards a longstanding challenging aerospace problem.
- The proposed controller explores and exploits the full aerodynamic nonlinearities of the extreme ends of the flight envelope of the aircraft to generate a control policy and its performance is not limited by any simplifications of the dynamics or aerodynamics.
- The proposed controller does not decompose into altitude and attitude control blocks but rather is a unified, generic block that is able to coordinate and command multi-actuator input for all desired maneuvers.
- The proposed controller does not require a precomputed feasible trajectory via a motion planning algorithm.

III. Background

This section outlines the underlying fundamental theory that frames the backbone of the Reinforcement Learning (RL) algorithm. The dynamics of the application is structured as a Markov Decision Process (MDP). Using this MDP, the experiential and reward-optimizing nature of RL is choreographed in such a way to generate the basis behind the idea of Q-learning which then allows for the realization of the Normalized Advantage Function (NAF) variant Q-learning algorithm.

A. Markov Decision Processes [19]

A *Markov Decision Process* (MDP) is defined as a tuple $\langle \mathcal{X}, \mathcal{U}, \mathcal{P}_{xx'}^u, \gamma, R \rangle$ where $\mathcal{X} \in \mathbb{R}^n$ is the fully observed n-dimensional state space of the process, $\mathcal{U} \in \mathbb{R}^m$ defines the m-dimensional action space of the process and $\mathcal{P}_{xx'}^u$, hereafter written as $\mathcal{P}$, is the state transition probability matrix that defines the Markovian transition dynamics of moving from a state x to a new state x' under an action u , drawn from $\mathcal{X}$ and $\mathcal{U}$, respectively.

$$\mathcal{P}_{xx'}^u = \mathbb{P}[X_{t+1} = x' | X_t = x, U_t = u] \quad (1)$$

The reward function $R \in \mathbb{R}^1$ denotes the expected reward for being in a state x by having taken an action u and $\gamma \in [0, 1]$ is the discount factor for future rewards. This represents a mechanism for a trade-off between immediate and future rewards.

For an MDP, the deterministic policy $\pi(u|x)$ is the distribution over actions given states:

$$\pi(u|x) = \mathbb{P}[U_t = u | X_t = x] \quad (2)$$

B. Model-Free Reinforcement Learning

Reinforcement Learning (RL) is a sub-type of machine learning in which an agent interacts with a possibly stochastic environment via trial-and-error experiences [20]. At every stage of interaction, the agent records observations of the environment and receives rewards from the environment for taking an action. In model-free RL, the environment is modeled as a black box, and therefore the agent has no knowledge of the dynamics of the environment. The goal of the agent in RL is to learn a policy that maximizes its total expected rewards via its interactions with the environment. The total expected reward is defined as:

$$R_t = \sum_{i=t}^T \gamma^i r(x_i, u_i) \quad \forall T < \infty \quad (3)$$

where R_t is the reward at time step t for $0 \leq t \leq T < \infty$ and $r(x_i, u_i)$ is the immediate reward for the i th state x_i and i th action u_i .

Since the RL process is modeled as an MDP, the underlying properties of the MDP can be utilized to define the *action-value* function $Q_\pi(x, u) \in \mathbb{R}^1$ as per [20] as:

$$Q_\pi(x, u) = \mathbb{E}_{u_t \sim \pi, x_t \sim \mathcal{P}} \left[\sum_{i=t}^T \gamma^i r(x_i, u_i) | x_t = x, u_t = u \right] \quad (4)$$

The *action-value* function $Q_\pi(x, u)$ is therefore the expected sum of all discounted rewards from a state x , achieved by taking an action u drawn from a policy $\pi(x, u)$.

C. Q-Learning Algorithm

The principle behind the Q-learning algorithm is to optimize the value of $Q_\pi(x, u)$, and hence $Q_\pi(x, u) = \max_u Q_\pi(x, u)$ where the optimal value $Q_\pi(x, u)$ can be estimated. The Q-learning algorithm utilizes the idea of value evaluation and improvement over successive iterations. For an arbitrarily initialized Q -value, an action u is sampled from a policy π . The subsequent reward r and state x' are then observed. The optimal value q_π is then the greedy maximal value over all feasible actions $u \in \mathcal{U}$ of the target Q -function ($Q'(x, u)$) given the next state x' . This is estimated via the Bellman Optimality Equation [19]:

$$q_\pi(x, u) = \mathbb{E}[r_t + \gamma \max_{u'} Q'(x', u') | x_t = x, u_t = u] \quad (5)$$

This maximal target value is then used to perform a one-step update by pushing the current Q -value towards this optimal Q -value by this amount with a step size $\alpha \in \mathbb{R}^1$. Thus the update equation is then:

$$Q(x_t, u_t) = Q(x_t, u_t) + \alpha [q_\pi(x, u) - Q(x_t, u_t)] \quad (6)$$

Typically the Q -Learning algorithm utilizes a look-up table to storing and updating the Q values associated with each state-action pair. However, this is not practical for the high dimensional state or action domains [21]. Instead, the Q values can be represented by a function approximator with weights θ that estimates the value of Q given a state x and an action u , i.e., $Q \approx Q^*(s, a; \theta_Q)$. Since the Q -lookup table is now represented as a Q -network, this Q -net is iteratively trained and the weights of this net can be optimized by minimizing a designated loss function $L(\theta_Q)$ which is a function of $Q(x, u; \theta_Q)$.

IV. Aircraft Simulation & Maneuvers

This section details the chosen aircraft and the fully observable simulation environment used in demonstrating the controller performance. Additionally, each selected aerobatic maneuver and its constrained objective/reward function are subsequently outlined.

A. Environment and Aircraft Modeling

The dynamics of a Fixed-Wing aircraft is described and simulated using FlightGear with a backend Flight Dynamics Machine (FDM) called JSBSim [22]. The JSBSim FDM utilizes the concepts of aircraft geometry, lift and drag forces, and the resultant moments, created by control surface actuation and environmental conditions, to simulate the motion of aircraft flight. The Equations of Motion (EoM) of a Fixed-Wing aircraft is comprised of translational and rotational nonlinear equations of a 6 degree of freedom (6 DOF) system about the 3 axes shown in Fig. (1a) below. This generates a 12-dimensional fully observable state space, positions and their rates, corresponding to the following vector of states:

$$x = \begin{bmatrix} X \\ Y \\ Z \\ v_X \\ v_Y \\ v_Z \\ \phi \\ \theta \\ \psi \\ p \\ q \\ r \end{bmatrix} \quad \left. \begin{array}{l} \left. \begin{array}{l} X \\ Y \\ Z \end{array} \right\} \text{linear position} \\ \left. \begin{array}{l} v_X \\ v_Y \\ v_Z \end{array} \right\} \text{linear velocity} \\ \left. \begin{array}{l} \phi \\ \theta \\ \psi \end{array} \right\} \text{angular position} \\ \left. \begin{array}{l} p \\ q \\ r \end{array} \right\} \text{angular velocity} \end{array} \right\} \quad (7)$$

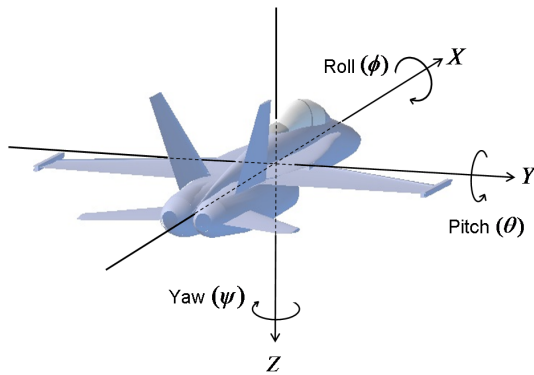
These state variables are primarily governed by the derivation of the nonlinear differential system $\dot{x} = f(x, u)$ as per [23] with emphasis on the extensive modeling of agile Fixed-Wing aircraft over the full flight envelope [11]. The propagation of the state variables is mainly driven by inputs to the control surface deflections and throttle which are captured by the

control input vector:

$$u = \begin{bmatrix} \delta_e \\ \delta_a \\ \delta_r \\ \delta_T \end{bmatrix} \quad (8)$$

where $\delta_i \in [-1, 1] \forall i \in \{a, e, r\}$ is the respective normalized commanded deflections from zero position in elevator, aileron and rudder while $\delta_T \in [0, 1]$ is the normalized commanded throttle command.

Corresponding to this formulation, the specific aircraft configuration used and modeled in JSBSim is the Mudry Cap10B aerobatic aircraft as shown in Fig. (1b). Developed in 1970 as a training aerobatic aircraft and with a status of "Advanced Production" in FlightGear, Cap10B has an FDM rating of 4 stars which corresponds to "FDM matches Pilot Operating Handbook (PoH) in 90% of configurations" [22]. For that reason, this aerobatic aircraft has been deemed to be of sufficient simulation fidelity for this study.



(a) Diagram of the body axes denoting the states of an aircraft



(b) Mudry Cap10B aircraft

Fig. 1 Overview of the chosen aircraft and its frame of reference

It is important to note that full knowledge of the kinematics and dynamics of the aircraft and its simulation environment is unnecessary. In fact, the controller treats the environment as a black box from which it gathers rewards. However, the maneuvers are described as a generically parametrized functions of the performance parameters of the aircraft. The performance factors for the Cap10B aircraft are detailed in the Appendix.

B. Executed Aerobatic Maneuvers with Reward Functions

This section details the specific maneuvers and all their pertaining execution information. The maneuvers are defined by an encapsulating reward function based on the desired state trajectory of the aircraft. This state trajectory is generically parametrized function for any arbitrarily-capable aircraft. In executing each maneuver, the utmost importance was placed on dynamic feasibility and optimality.

In defining the trajectory and reward function of the selected maneuvers, we found that it was more efficient and favorable to define each maneuver by a static starting condition, its rates of change and a final static condition on the states for each stage of motion. In doing so, we can then parametrize the maneuver according to the aircraft's measured performance factors, and allow for less constrained and more organic learning. Additionally, by defining the desired states as constant, the MDP formulation remains nonstationary.

1. Stabilize to Steady Level flight

While steady level flight is not an aerobatic maneuver, this flight primitive is often used as a transitional maneuver. Flying straight and level dictates a constant zero of the roll and pitch positions as well as maintaining a constant altitude and heading about an arbitrary operating point.

For steady level flight, R^{SL} , we seek to maximize a constrained reward function that minimizes time and the distance between the actual and desired reference states :

$$R^{SL} = - \sum_{t=t_0}^{t_f} [(\Phi_t^{SL} - \Phi_{ref}^{SL})^T W_{SL} (\Phi_t^{SL} - \Phi_{ref}^{SL}) + w_t t] \quad (9)$$

where $\Phi_t = [Z, \phi, \theta, \psi, p, q, r]_{time=t}^T$ is the vector of selected states at time t , $\Phi_{ref} = [Z_{ref} = Z_{t=0}, \phi_{ref} = 0^\circ, \theta_{ref} = 0^\circ, \psi_{ref} = \psi_{t=0}, p_{ref} = 0, q_{ref} = 0, r_{ref} = 0]^T$, $W_{SL} \in \mathbb{R}^{7 \times 7}$ is the positive diagonal matrix of weights, and w_t is the penalty associated with time. v_m is the minimum maneuver operating speed of the aircraft based on its maximum load factor and maximum level speed.

Additionally, $R_t^{SL} = 1$ if $(\Phi_t^{SL} - \Phi_{ref}^{SL}) \leq tol$ for some time t .

2. Slow Roll

A Slow Roll is a straight line maneuver in which the aircraft performs a constant rate controlled roll about its X -axis. The goal of the maneuver is to complete a full 360° rotation about the X -axis as fast as possible while maintaining a constant altitude and constant yaw angle. The Slow Roll is a popular aerobatic training maneuver, and while it is used very little in practice, we use it here to demonstrate the skill and versatility of the proposed controller to operate on the extreme ends of the flight regime.

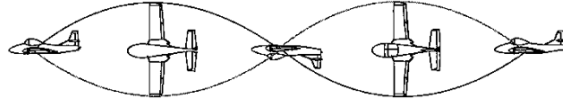


Fig. 2 Slow Roll Maneuver

The reward function of the Slow Roll, R^{SR} , is as described as follows:

$$R^{SR} = - \sum_{t=t_0}^{t_f} [(\Phi_t^{SR} - \Phi_{ref}^{SR})^T W_{SR} (\Phi_t^{SR} - \Phi_{ref}^{SR}) + w_t t] \quad (10)$$

subject to $v_X|_{t=0} \geq v_m$, $t_f \geq t_{min}$ and $\phi_{t_f} = 0$

where $\Phi_t^{SR} = [p, q, r]_{time=t}^T$ is the vector of angular velocities at time t , $\Phi_{ref}^{SR} = [p_{ref} = \dot{\phi}_{max}, q_{ref} = 0, r_{ref} = 0]^T$, $W_{SR} \in \mathbb{R}^{3 \times 3}$ is the positive diagonal matrix of weights and w_t is the penalty conferred on time. $\dot{\phi}_{max}$ is the maximum roll rate of the aircraft, and t_{min} is the minimum time for execution of the Slow Roll based on the max roll rate of the aircraft.

3. Knife Edge

The objective of the Knife Edge maneuver is to fly along a straight path with wings rolled to 90° as shown in Fig. (3). The maneuver involves sufficient aileron deflection to either the left or the right to roll and maintain a 90° angle about the X -axis of the aircraft with sufficient pitch upwards to ensure enough lift generation to maintain a constant altitude. Knife Edge maneuvers are commonly used for flying in gaps between obstacles that are narrower than the wingspan of the aircraft.

The state dependent cumulative reward function of the Knife Edge maneuver, R^{KE} , is described as the following constrained function profile:

$$R^{KE} = - \sum_{t=t_0}^{t_f} [(\Phi_t^{KE} - \Phi_{ref}^{KE})^T W_{KE} (\Phi_t^{KE} - \Phi_{ref}^{KE}) + w_\theta^{KE} (\theta_t < 0)] \quad (11)$$

subject to $v_X|_{t=0} \geq v_m$

where $\Phi_t = [\phi, p, q, r]_{time=t}^T$ is the vector of angular velocities and the roll state at time t , $\Phi_{ref} = [\phi_{ref} = 90^\circ, p_{ref} = 0, q_{ref} = 0, r_{ref} = 0]^T$ is the vector of desired reference states for the Knife Edge maneuver, $W_{KE} \in \mathbb{R}^{3 \times 3}$ is the positive diagonal matrix of weights, and w_θ^{KE} is the penalty conferred on a negative pitch angle.

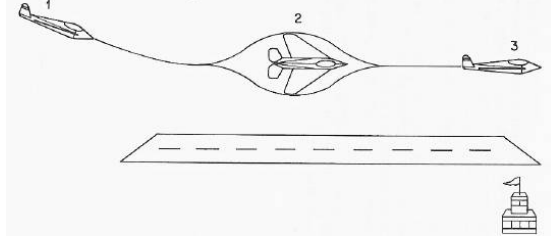


Fig. 3 Knife Edge Maneuver

Additionally, $R_t^{KE} + 1$ if $(\Phi_t^{KE} - \Phi_{ref}^{KE}) \leq tol$ for some time t .

V. Methodology

This section comprehensively details the proposed Deep RL controller to enable learning of different aerobatic maneuvers. Using the NAF framework, the agent learns, via reward maximization, a policy controller $\pi_\theta(u|x)$ that maps the fully observable environment's states x to actions u . The controller exploits the learnt behavior developed during its trial-and-error exploration of the 12-dimensional state space of the aircraft and its 4-dimensional action space during the training period.

During training, additive noise, v_t , is injected into the action space to enable perturbed behaviors that allow for exploration of the state space. The proposed NAF controller treats the simulation environment as black box from which it observes the resultant states of noisy control inputs u_t . These states are then injected into a reward calculator module that contains maneuver specific information. On each iteration, the resulting reward value is then used to update the parameters of the controller via Policy Gradient Descent (PGD) optimization of a candidate loss function $L(\theta_Q)$ associated with the neural net as per Section III.C.

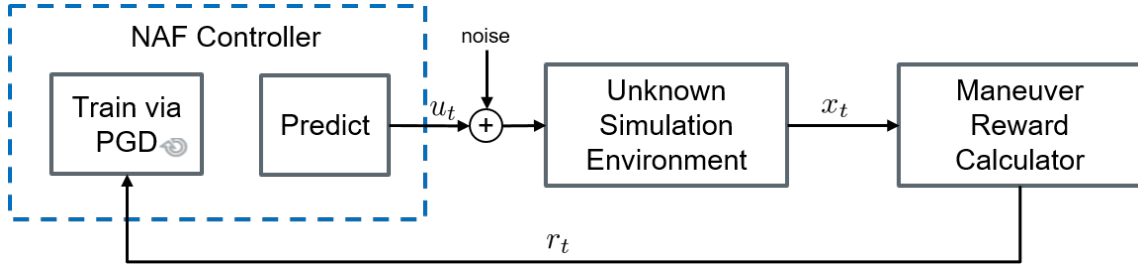


Fig. 4 Overview of NAF controller architecture

A. Normalized Advantage Function (NAF) controller

One of the primary contributions of this paper lies in the simple unified formulation of the NAF controller with its single neural net which predicts and coordinates a multi-action state space of size m from a large multidimensional observation/state space $x_t \in \mathbb{R}^n$. Additionally, this single net structure is able to predict a scalar value quantity $V(x_t) \in \mathbb{R}^1$ and a scalar action-value quantity $Q(x_t, u_t) \in \mathbb{R}^1$ for policy optimization, unlike other variant RL algorithms.

As described in [24], the NAF algorithm utilizes the concept of the value function $V(x_t)$ and the Advantage function $A_\pi(x_t, u_t)$ to calculate the action-value function $Q(x_t, u_t)$. Intuitively, the Advantage function $A_\pi(x_t, u_t)$ speaks to how much better taking an action u_t from a selected max value of $Q(x_t, u_t)$ is over following the current policy π from the current state x_t . The value function and the Advantage function are respectively defined as:

$$V(x_t) = \mathbb{E}_{r_t, x_t \sim \epsilon, u_t \sim \pi} [r_t | x_t] = \max_u [Q(x_t, u_t) | x_t = x, u_t = u] \quad (12)$$

$$A_\pi(x_t, u_t) = Q_\pi(x_t, u_t) - V(x_t) \quad (13)$$

The NAF neural net design outputs a scalar quantity for $V(x_t)$ based on the state x_t , a value $A_\pi(x_t, u_t)$ as well as the predicted control actions $\mu(x_t)$. The value function and the Advantage function combine to calculate the Q -value as

shown in Fig. (5).

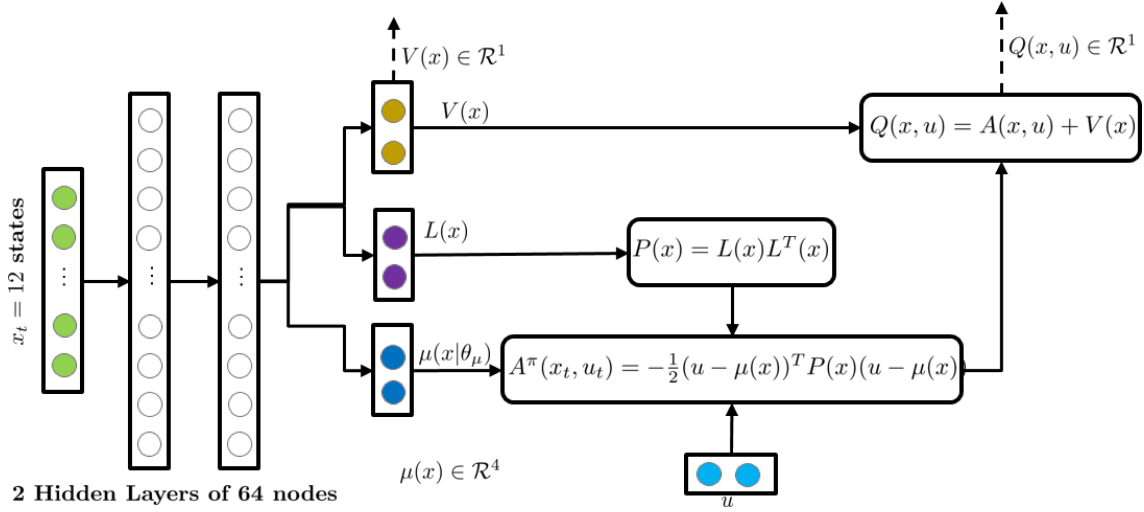


Fig. 5 Architecture of NAF neural network

The following gives a concise and mathematically detailed outline of the NAF neural net shown in Fig. (5), from [24], above:

$$\begin{aligned}
 s(x) &= \Re(x) \\
 h_1(x) &= g_1(\vartheta_{h_1}^T s(x) + b_{h_1}) \\
 h_2(x) &= g_2(\vartheta_{h_2}^T h_1(x) + b_{h_2}) \\
 L'(x) &= \vartheta_L^T h_2(x) + b_L \\
 L(x) &= \{f : L'(x) \in \mathbb{R}^{m^2 \times 1} \rightarrow \mathbb{R}^{m \times m}\} \\
 L(x) &= \text{tril}(L(x), -1) + \exp(\text{diag}(L(x))) \\
 P(x) &= L(x)L^T(x) \\
 V(x) &= \vartheta_v^T h_2(x) + b_v \\
 \mu(x) &= g_\mu(\vartheta_\mu^T h_2(x) + b_\mu) \\
 A_\pi(x, u) &= -\frac{1}{2}(u - \mu(x))^T P(x)(u - \mu(x)) \\
 Q_\pi(x, u) &= A_\pi(x, u) + V(x)
 \end{aligned}
 \tag{14}$$

where:

- $\Re(z)$ is the batch normalization function for some input z .
- $h_i \forall i \in \{1, 2\}$ are the first and second hidden layers of the net, each of 64 nodes.
- $g_i \forall i \in \{1, 2, \mu\}$ is the $\tanh(z)$ activation function for some input variable z for the first and second hidden layers, and the μ output layer respectively.
- $\vartheta_i \forall i \in \{h_1, h_2, L', \mu, v\}$ are the weights pertaining to the corresponding first and second hidden layers, and V and μ output layers of the neural net.
- $b_i \forall i \in \{h_1, h_2, L', \mu, v\}$ are the bias weights pertaining to the corresponding first and second hidden layers, and V and μ output layers of the neural net.
- $L'(x) \in \mathbb{R}^{m^2}$ is the linear output from the neural net that is used to form the Advantage Function $A(x, u)$.
- $P(x) \in \mathbb{R}^{m \times m}$ is the isomorphic positive definite matrix formed by reshaping $L'(x) \rightarrow \mathbb{R}^{m \times m}$ and then extracting its lower triangular matrix and exponentiating its diagonal.
- $\mu(x) \in \mathbb{R}^4$ are the predicted actions from the net.

In order to create the controller, it is necessary to train the NAF Policy Q-net with state data. During training, the states $x \in \mathcal{X}$ are sampled from the simulated environment $\mathcal{P}_{xx'}^{\hat{u}}$, given noisy actions $\hat{u} = u + v$, where v is the additive

correlated colored noise generated by the Ornstein–Uhlenbeck (OU) process:

$$v_k = v_{k-1} + \theta_v(\hat{\mu} - v_{k-1}) + \sigma \epsilon_k \quad (17)$$

$\epsilon_k \sim \mathcal{N}(0, \sigma^2)$ is a sequence of independent random numbers generated from a normal distribution with mean of 0 and variance σ^2 , $\hat{\mu}$ is the mean of the correlated process and θ_v is a lag correlation coefficient of the process.

Using these observed states x , a reward value is calculated based on the desired maneuver. This reward value is then used to update the corresponding value $Q(x, u)$ which is then used to optimize the net.

B. Maneuver Reward Calculator

The maneuver reward calculator is the module that returns the instantaneous reward, r_t , at some time t according to the corresponding maneuver as detailed in section IV.B.

$$\begin{aligned} r_t^{SL} &= -(\Phi_t^{SL} - \Phi_{ref}^{SL})^T W_{SL}(\Phi_t^{SL} - \Phi_{ref}^{SL}) - w_t t + \{1 \text{ if } (\Phi_t^{SL} - \Phi_{ref}^{SL}) \leq tol\} \\ r_t^{SR} &= -(\Phi_t^{SR} - \Phi_{ref}^{SR})^T W_{SR}(\Phi_t^{SR} - \Phi_{ref}^{SR}) - w_t t \\ r_t^{KE} &= -(\Phi_t^{KE} - \Phi_{ref}^{KE})^T W_{KE}(\Phi_t^{KE} - \Phi_{ref}^{KE}) - w_{\theta}^{KE}(\theta_t < 0) + \{1 \text{ if } (\Phi_t^{KE} - \Phi_{ref}^{KE}) \leq tol\} \end{aligned}$$

subject to conditions on $R^M \forall M \in \{SL, SR, KE\}$.

C. Policy Gradient Descent (PGD) Optimization

Consider a candidate smooth l_1 loss function:

$$L(\theta_Q) = \frac{1}{n} \sum_t z_t \quad (18)$$

where z_t is given by:

$$z_t = \begin{cases} 0.5[Q(x, u|\theta_Q) - y_t]^2 & \text{if } |Q(x, u|\theta_Q) - y_t| \leq 1; \\ |Q(x, u|\theta_Q) - y_t| - 0.5 & \text{otherwise.} \end{cases}$$

and

$$y_t = r_t + \gamma^t V'(x_{t+1}|\theta_{Q'}) \quad (19)$$

where $\gamma = 0.99$ is the discount factor, r_t is the time t specific reward from R_t , and $V'(x_{t+1}|\theta_{Q'})$ is the value function of the current state, based on its value with respect to the goal state, predicted from a target Q -net ($Q'(x, u)|\theta_{Q'}$) at time step t .

For training in the current implementation, the policy is twice optimized and updated via gradient descent during experience replay. The gradient of the loss function is calculated as:

$$\nabla_{\theta_Q} L(\theta_Q) = E_{s \sim \epsilon} [\nabla_{\theta_Q} z_t \nabla_{\theta_Q} Q(x, u)] \quad (20)$$

Using this derivative, the weights of the policy are updated as:

$$\theta_Q(t+1) = \theta_Q(t) + \alpha(t) \times \eta \times \nabla_{\theta_Q} L(\theta_Q) \quad (21)$$

where η is the loss rate, and $\alpha(t) \in \mathbb{R}^1$ is the scalar function of decaying step size.

VI. Simulation Results

In this section, we demonstrate the performance of the proposed Deep RL controller via two illustrative aerobatic maneuvers: Slow Roll and Knife Edge. In particular, we demonstrate the execution of each maneuver, the computational labor of the controller with respect to the time taken and the number of episodes required for convergence, as well as a sketch of the explored regions of the flight envelope. We also show that we are able to stitch together maneuver primitives to: a) execute each complete aerobatic maneuver and, b) to execute an aerobatic maneuver sequence.

It should be noted that training of the flight controller was undertaken by randomizing the initial episodic aircraft state over the following flight operating conditions:

- (i) Flight altitudes in the operating range of 2500-7500 ft.
- (ii) Initial calibrated airspeeds ranging from 10 knots above stall speed to max level speed.

A. Training Results

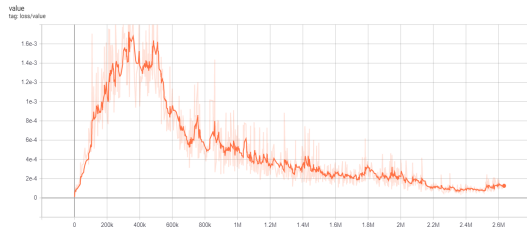
In this section, we show that the proposed controller converges to each respective aerobatic maneuver target behavior as it attempts to maximize its cumulative reward and converges to a maximum reward. We also give the execution time and number of episodes required for this convergence given an intel i7-7700HQ CPU @2.80 Hz with 32 GB RAM.

Table 1 below shows the average episodic reward, the total number of episodes taken for convergence, and the total execution time taken for convergence during training for each maneuver outlined in Section IV.B.

Table 1 Execution time and results for training for each maneuver

Aerobatic Maneuver	Average Reward	Number of episodes/Max episode length	Learning Rate (γ)	Execution Time (s)
Steady Level Flight	40.0	8303/1000	0.0005	16624.712
Slow Roll	-7.5	1431/150	0.001	696.638
Knife Edge	29.0	8900/81	0.001	9080.387

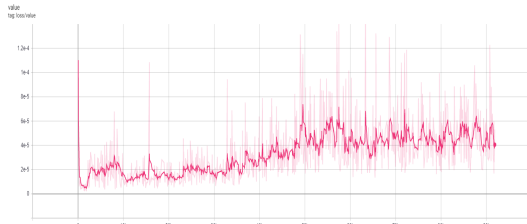
These results show that during training, the controller is able to learn the selected maneuvers on the order of hours. The maneuver is considered learnt when the training procedure produces a steady average episodic reward over 500 episodes. Figure 6 shows that after an initial increase, the controller tries to minimize the loss function over the course of training while the reward function is maximized and converges to a steady episodic value, which is stated above for each maneuver.



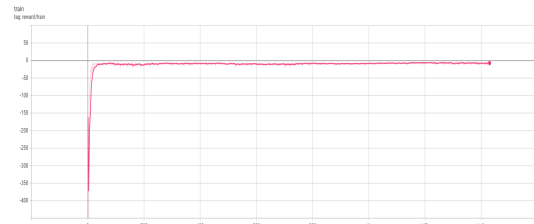
(a) Smoothed loss across updates for Stabilize maneuver



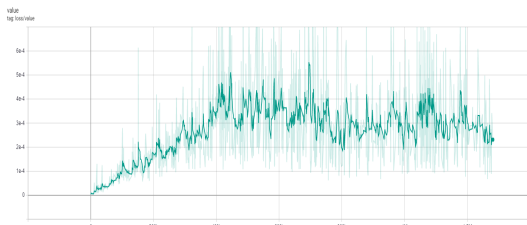
(b) Smoothed episodic reward for Stabilize maneuver



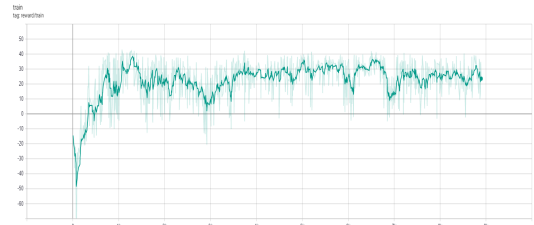
(c) Smoothed loss across updates for Slow Roll maneuver



(d) Smoothed episodic reward for Slow Roll maneuver



(e) Smoothed loss across updates for Knife Edge maneuver



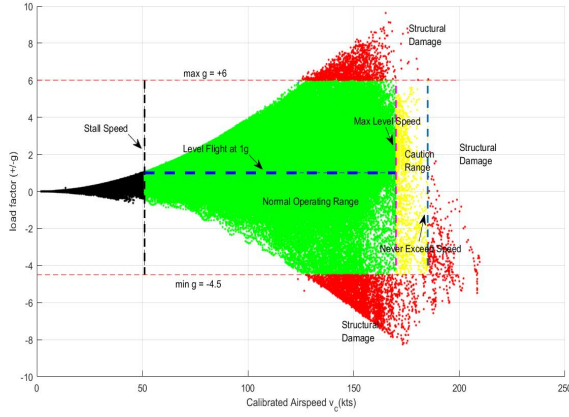
(f) Smoothed episodic reward for Knife Edge maneuver

Fig. 6 Loss function and episodic rewards gained during training for each demonstrated maneuver

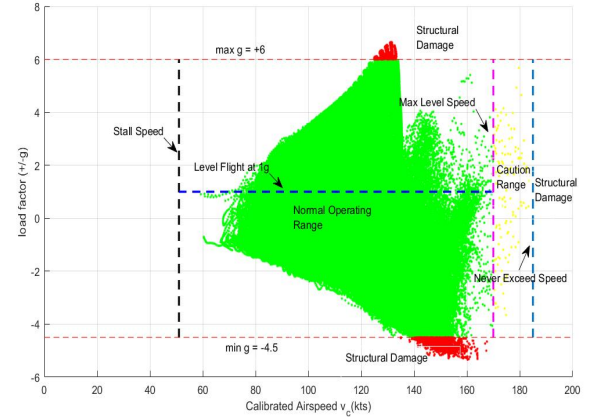
B. Flight Envelope Exploration during Training

One of the principal contributions of this paper is the emphasis on exploring the full nonlinear flight envelope of the aircraft to enact optimal maneuvering behavior. The results below demonstrate the proposed controller is able to utilize

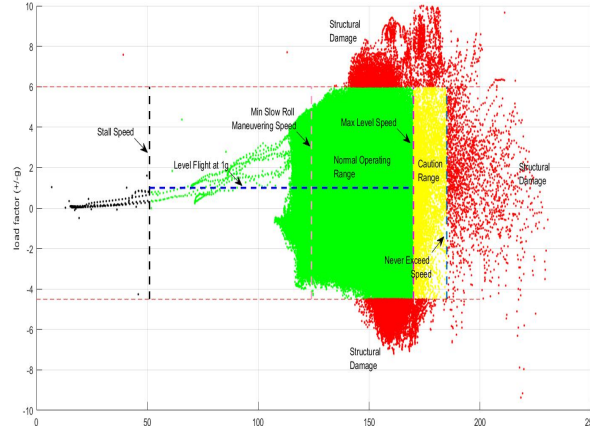
information on the edges and beyond the flight envelope to converge to the best performance for each maneuver.



(a) Explored V-n envelope during Stabilize training



(b) Explored V-n envelope during Knife Edge training



(g) Explored V-n envelope during Slow Roll training

Fig. 7 Explored regions of the flight envelope during maneuver training

During the Stabilize to Level flight maneuver, the controller explores the full flight envelope of the aircraft, even the danger regions of structural damage. However, during the Knife Edge and Slow Roll maneuver, the controller does not explore much below the minimum maneuvering speed owing to the hardcoded restrictive condition on the initial airspeed.

C. Maneuver Results

In this section, we concatenate the steady level flight controller with each aerobatic maneuver controller to demonstrate the performance of a full aerobatic maneuver from level flight to level flight. Please note that all simulations are Software-In-The-Loop real time simulations in FlightGear with JSBSim FDM.

1. Slow Roll

Firstly, we show that the aircraft is able to perform a slow roll maneuver beginning from level flight and ending in level flight as shown in Fig. (8). Figure (11) shows the altitude, angular positions and rates of the cap10B aircraft during the Slow Roll maneuver. Since the controller was trained at varying flight conditions, the aircraft was able to perform a complete roll in approx. 3 seconds with a constant roll rate of 140 deg/s which corresponds to a rate slightly less than the max roll rate of the aircraft. Initially, the controller pitches the aircraft up to approximately 5° , in order to

minimize the overall drop in altitude, then utilizes both elevator and rudder control inputs to stabilize the roll such that deviation in the pitch and yaw angles are minimized at the end of the roll. At the end of the roll, the aircraft has lost an approximate average of 150 ft in altitude. The aircraft then stabilizes to steady level flight. The aircraft is considered stable if the wings are level and the angular rates remain approximately zero for 5 consecutive seconds.

During the stabilizing segment of the maneuver, we observe that the pitch and roll angles converge to a steady state within 2° of their commanded zero position across all test runs. The yaw angle converges to within 6° of its initial heading. All corresponding rates converge to zero. The results of the altitude show that there is a dip in altitude during the stabilizing maneuver and then a rise. This dip may be because the stabilizing controller was cold-started after the roll was finished.

2. Knife Edge

Secondly, we demonstrate the Knife Edge maneuver starting from steady level flight and ending in steady level flight. In Figure (12), we observe that, during the Knife Edge maneuver, the controller was able to command and maintain a 90° roll angle for over 6 seconds. However, the pitch profile steadily deteriorated during the maneuver. The pitch angle decreased significantly. We reason that this is because the rates of change of pitch were commanded to be zero throughout the maneuver. While this might be the ideal, we reason that these rates should be left free in order to compensate for deviation in the pitch angle that arise as a result of the g-forces acting on the aircraft.

On stabilization, the controller levels the orientation of the aircraft by driving the angular rates to zero. With respect to the altitude, the controller seeks to return to its initial altitude, therefore the overall drop in altitude throughout the maneuver is less than 30 ft.

D. Maneuver Sequence

Thirdly, we demonstrate, in this section, the ability to sequentially stitch together the different maneuvers to generate a maneuver sequence. In Fig. (10), we demonstrate a Knife Edge and Slow Roll sequence combination ending in steady level flight.

At the end of the sequence of maneuvers, the aircraft has lost approximately 350 ft of altitude. We reason that this is because the aircraft has lost a significant amount of energy during the Slow Roll maneuver and lost even more while executing the cold-started Knife Edge maneuver.

VII. Conclusion

In this paper, a Deep RL based controller has been proposed for enabling aerobatic maneuvering for capable Fixed-Wing aircraft. The proposed controller utilized the Normalized Advantage Function with a single Q-net architecture to formulate a unified control scheme that is able to coordinate and command full multi-actuator authority respective of its magnitude limits. The performance and computational efficiency of the proposed controller have been tested and demonstrated with illustrative aerobatic maneuvers in a real time simulation setup. As for future work, the proposed controller will seek to incorporate a comprehensive definition and conceptualization of aircraft energy management as defined by its load limit and turn rates of the aircraft. Additionally, we intend to train and deploy the proposed controller on hardware onboard a small-scale UAV for real testing.

Acknowledgements

The authors would like to acknowledge that this work is supported by Rockwell Collins/ Collins Aerospace. Additionally the authors would like to acknowledge Phil Baldwin whose personal experience performing different aerobatic maneuvers we leveraged to provide theoretic insights on this project.

Appendix

Table 2 Aircraft Performance Properties [25]

Parameter	Value	Units
Stall speed	51	knots
Min Slow Roll speed	124	knots
Cruise speed	135	knots
Max Roll rate	150	deg/s
Max Level speed	170	knots
Never Exceed Speed	185	knots
Ceiling Altitude	10000	ft
Load Limit (−/+)	−4/+6	g

References

- [1] Selig, M. S., “Real-Time Flight Simulation of Highly Maneuverable Unmanned Aerial Vehicles,” *Journal of Aircraft*, Vol. 51, No. 6, 2014, pp. 1705–1725. doi:10.2514/1.C032370, URL <https://doi.org/10.2514/1.C032370>.
- [2] Green, W. E., and Oh, P. Y., “A MAV that flies like an airplane and hovers like a helicopter,” *Proceedings, 2005 IEEE/ASME International Conference on Advanced Intelligent Mechatronics.*, 2005, pp. 693–698. doi:10.1109/AIM.2005.1511063.
- [3] Frank, A., McGrew, J., Valenti, M., Levine, D., and How, J., “Hover, Transition, and Level Flight Control Design for a Single-Propeller Indoor Airplane,” *AIAA Guidance, Navigation and Control Conference and Exhibit*, 2007, pp. 1–18. doi:10.2514/6.2007-6318, URL <https://arc.aiaa.org/doi/abs/10.2514/6.2007-6318>.
- [4] Roberts, J. W., Cory, R., and Tedrake, R., “On the controllability of fixed-wing perching,” *2009 American Control Conference*, 2009, pp. 2018–2023. doi:10.1109/ACC.2009.5160526.
- [5] Moore, J., Cory, R., and Tedrake, R., “Robust post-stall perching with a simple fixed-wing glider using LQR-Trees,” *Bioinspiration & Biomimetics*, Vol. 9, No. 2, 2014, p. 025013. doi:10.1088/1748-3182/9/2/025013, URL <https://doi.org/10.1088%2F1748-3182%2F9%2F2%2F025013>.
- [6] Li, B., Zhou, W., Sun, J., Wen, C.-Y., and Chen, C.-K., “Development of Model Predictive Controller for a Tail-Sitter VTOL UAV in Hover Flight,” *Sensors (Basel, Switzerland)*, Vol. 18, No. 9, 2018, p. 2859. doi:10.3390/s18092859, URL <https://www.ncbi.nlm.nih.gov/pubmed/30200199>[pmid].
- [7] Levin, J. M., Paranjape, A., and Nahon, M., “Agile fixed-wing UAV motion planning with knife-edge maneuvers,” *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 114–123. doi:10.1109/ICUAS.2017.7991475.
- [8] Levin, J. M., Nahon, M., and Paranjape, A. A., “Aggressive Turn-Around Manoeuvres with an Agile Fixed-Wing UAV,” *IFAC-PapersOnLine*, Vol. 49, No. 17, 2016, pp. 242 – 247. doi:<https://doi.org/10.1016/j.ifacol.2016.09.042>, URL <http://www.sciencedirect.com/science/article/pii/S2405896316315142>, 20th IFAC Symposium on Automatic Control in AerospaceACA 2016.
- [9] Levin, J., Paranjape, A., and Nahon, M., “Motion Planning for a Small Aerobatic Fixed-Wing Unmanned Aerial Vehicle,” *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 8464–8470. doi:10.1109/IROS.2018.8593670.
- [10] Bulka, E., and Nahon, M., “Automatic Control for Aerobatic Maneuvering of Agile Fixed-Wing UAVs,” *Journal of Intelligent & Robotic Systems*, Vol. 93, No. 1, 2019, pp. 85–100. doi:10.1007/s10846-018-0790-z, URL <https://doi.org/10.1007/s10846-018-0790-z>.

- [11] Khan, W., and Nahon, M., "Modeling dynamics of agile fixed-wing UAVs for real-time applications," *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2016, pp. 1303–1312. doi:10.1109/ICUAS.2016.7502599.
- [12] Ure, N. K., and Inalhan, G., "Autonomous Control of Unmanned Combat Air Vehicles: Design of a Multimodal Control and Flight Planning Framework for Agile Maneuvering," *IEEE Control Systems Magazine*, Vol. 32, No. 5, 2012, pp. 74–95. doi:10.1109/MCS.2012.2205532.
- [13] Sobolic, F., "Agile Flight Control Techniques for a Fixed-Wing Aircraft," Master's thesis, MIT, Cambridge, MA, 2009.
- [14] Arulkumaran, K., Deisenroth, M. P., Brundage, M., and Bharath, A. A., "Deep Reinforcement Learning: A Brief Survey," *IEEE Signal Processing Magazine*, Vol. 34, 2017, pp. 26–38.
- [15] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., and Hassabis, D., "Mastering the game of Go without human knowledge," *Nature*, Vol. 550, 2017, pp. 354–. URL <http://dx.doi.org/10.1038/nature24270>.
- [16] Hwangbo, J., Sa, I., Siegwart, R., and Hutter, M., "Control of a Quadrotor With Reinforcement Learning," *IEEE Robotics and Automation Letters*, Vol. PP, 2017, pp. 1–1.
- [17] Abbeel, P., Coates, A., Quigley, M., and Ng, A. Y., "An application of reinforcement learning to aerobatic helicopter flight," *Advances in neural information processing systems*, 2007, pp. 1–8.
- [18] Abbeel, P., Coates, A., and Ng, A. Y., "Autonomous helicopter aerobatics through apprenticeship learning," *The International Journal of Robotics Research*, Vol. 29, No. 13, 2010, pp. 1608–1639.
- [19] Sutton, R. S., and Barto, A. G., *Introduction to Reinforcement Learning*, 1st ed., MIT Press, Cambridge, MA, USA, 1998.
- [20] Silver, D., "Lecture notes in Reinforcement Learning, Advanced Topics 2015," , 2015.
- [21] Papis, J., and Lagoudakis, M. G., "Reinforcement learning in multidimensional continuous action spaces," *2011 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, 2011, pp. 97–104. doi:10.1109/ADPRL.2011.5967381.
- [22] Berndt, J. S., *JSBSim An open source, platform-independent, flight dynamics model in C++*, JSBSim, 2011.
- [23] Hull, D., *Fundamentals of airplane flight mechanics*, Springer Berlin Heidelberg, 2007. doi:10.1007/978-3-540-46573-7.
- [24] Gu, S., Lillicrap, T., Sutskever, I., and Levine, S., "Continuous Deep Q-learning with Model-based Acceleration," *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48*, JMLR.org, 2016, pp. 2829–2838. URL <http://dl.acm.org/citation.cfm?id=3045390.3045688>.
- [25] Robin, C., *AIRCRAFT FLIGHT MANUAL AND PILOT OPERATING HANDBOOK- CAP10b*, APEX à DYN'AVIATION, ????
- [26] Huber, P. J., "Robust Estimation of a Location Parameter," *Ann. Math. Statist.*, Vol. 35, No. 1, 1964, pp. 73–101. doi: 10.1214/aoms/1177703732, URL <https://doi.org/10.1214/aoms/1177703732>.
- [27] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D., "Continuous control with deep reinforcement learning," *iclr*, Vol. abs/1509.02971, 2015. URL <http://arxiv.org/abs/1509.02971>.
- [28] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M. A., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D., "Human-level control through deep reinforcement learning," *Nature*, Vol. 518, No. 7540, 2015, pp. 529–533. URL <http://dblp.uni-trier.de/db/journals/nature/nature518.html#MnihKSRVBGRFOPB15>.
- [29] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M., "Playing Atari with Deep Reinforcement Learning," , 2013.
- [30] *Aerobatic Maneuvering of Miniature Air Vehicles Using Attitude Trajectories*, 2012. doi:10.2514/6.2008-7257, URL <https://arc.aiaa.org/doi/abs/10.2514/6.2008-7257>.
- [31] Gavrilits, V., Mettler, B., and Feron, E., "Human-Inspired Control Logic for Automated Maneuvering of Miniature Helicopter," *Journal of Guidance, Control, and Dynamics*, Vol. 27, No. 5, 2004, pp. 752–759. doi:10.2514/1.8980, URL <https://doi.org/10.2514/1.8980>.

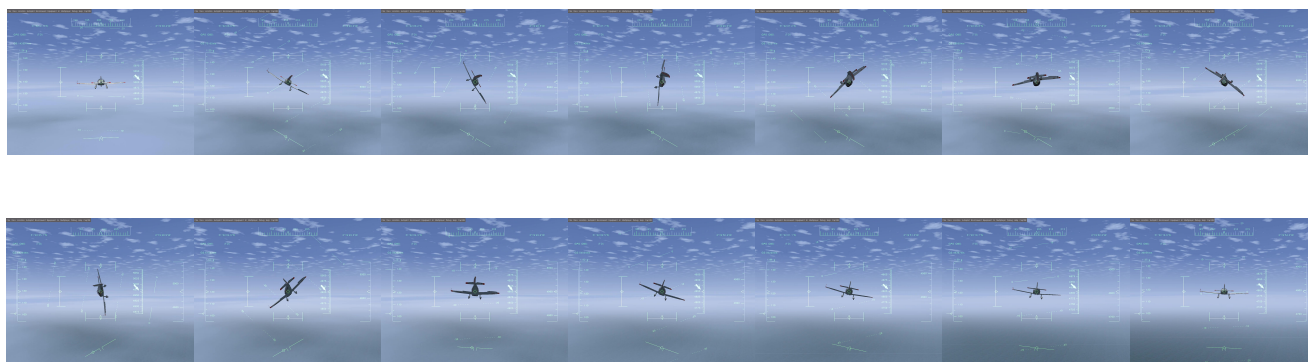


Fig. 8 Cap10B aircraft performing Slow Roll in FlightGear

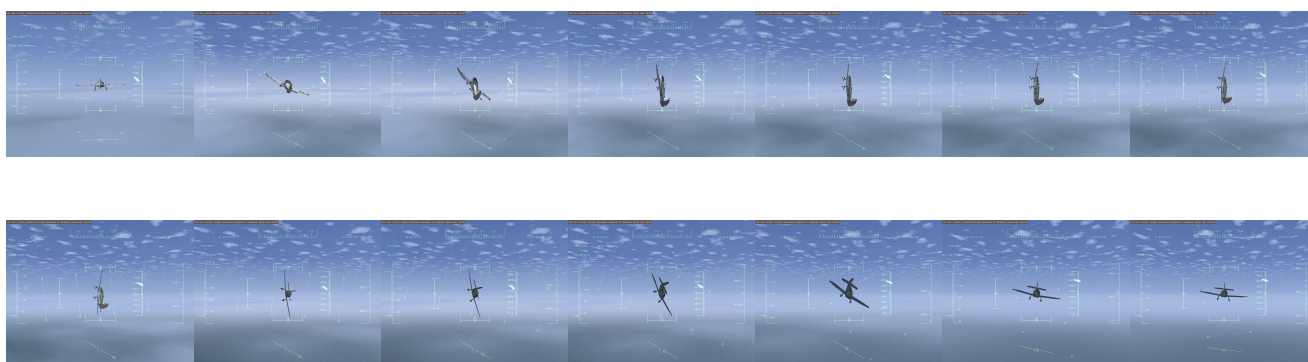


Fig. 9 Cap10B aircraft performing Knife Edge maneuver in FlightGear

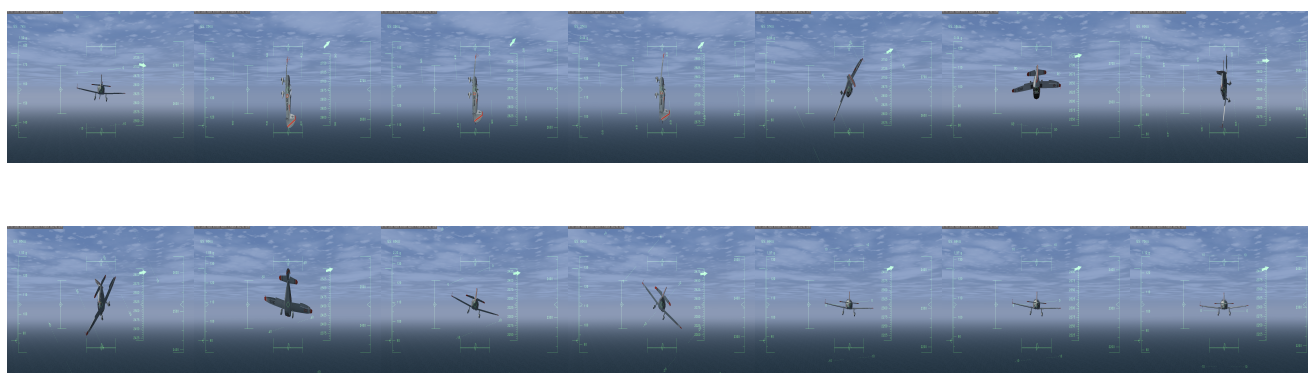
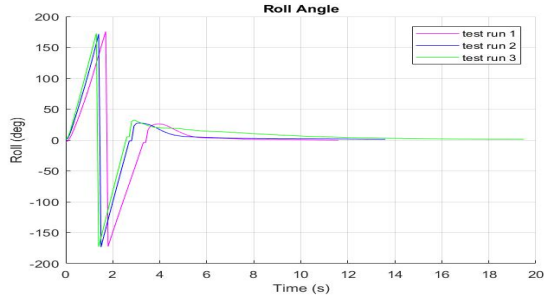
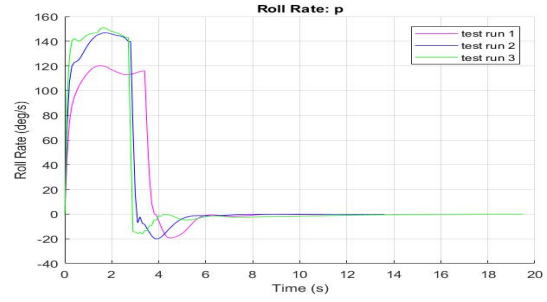


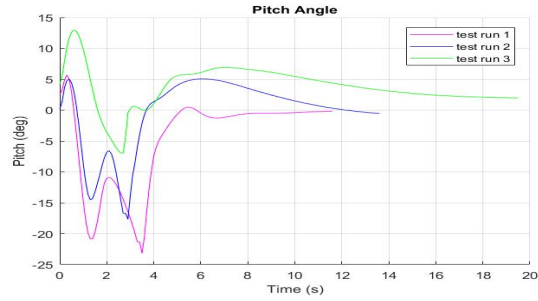
Fig. 10 Knife Edge- Slow Roll combination maneuver sequence



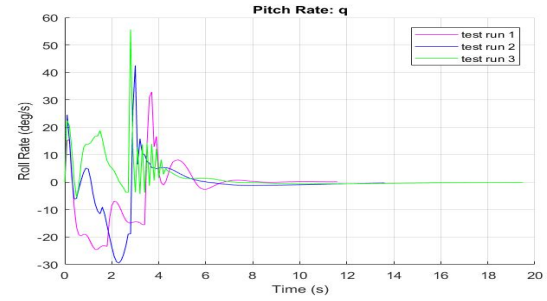
(a) Roll angle, ϕ , of the aircraft



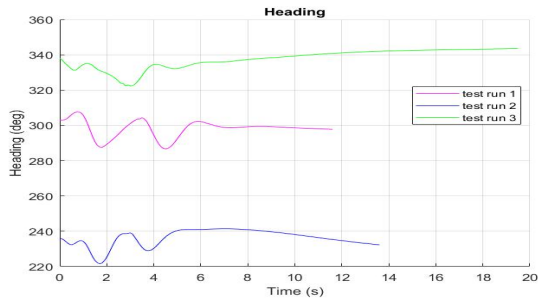
(b) Roll rate, p , of the aircraft



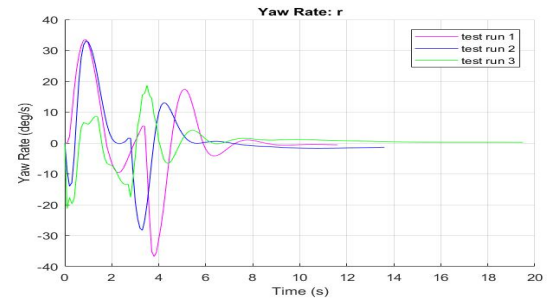
(c) Pitch angle, θ , of the aircraft



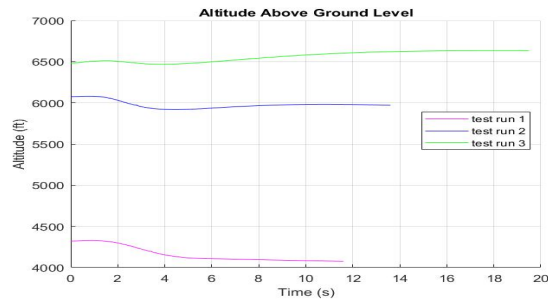
(d) Pitch rate, q , of the aircraft



(e) Heading, ψ , of the aircraft

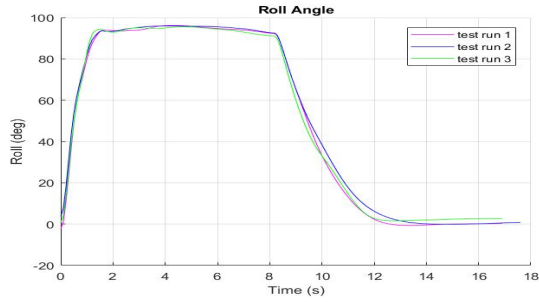


(f) Yaw rate, r , of the aircraft

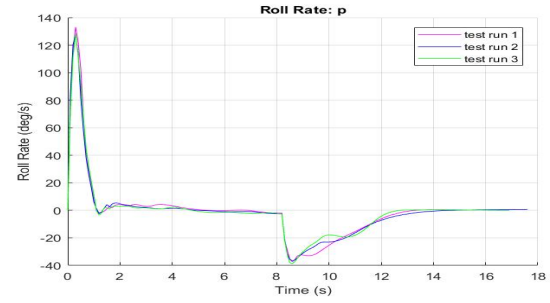


(g) Altitude of the aircraft

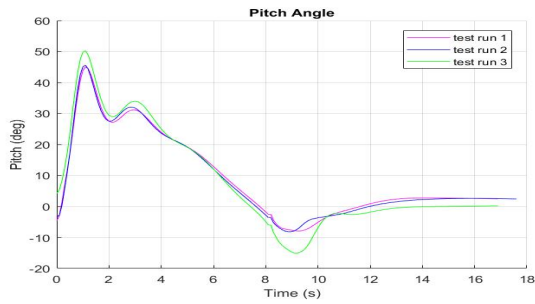
Fig. 11 States for the aircraft during the Slow Roll maneuver for test runs with varying initial conditions



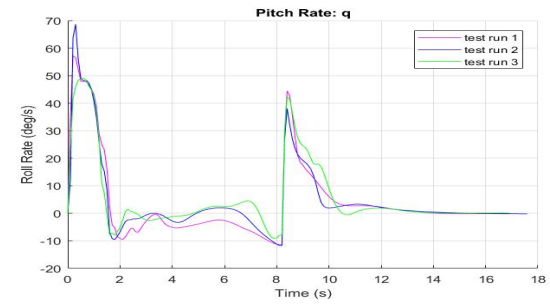
(a) Roll angle, ϕ , of the aircraft



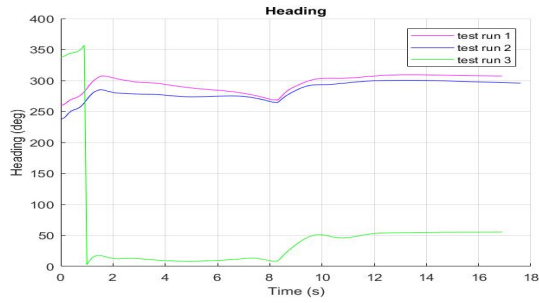
(b) Roll rate, p , of the aircraft



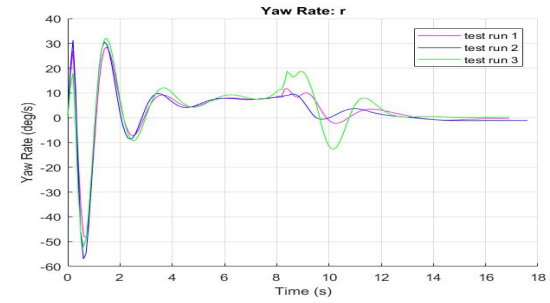
(c) Pitch angle, θ , of the aircraft



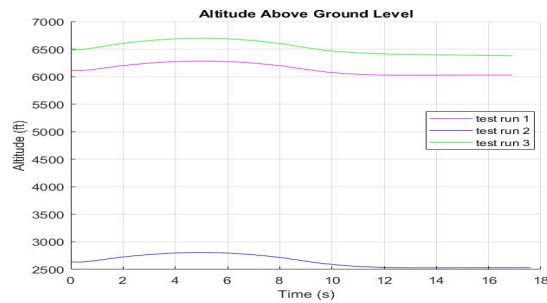
(d) Pitch rate, q , of the aircraft



(e) Heading, ψ , of the aircraft

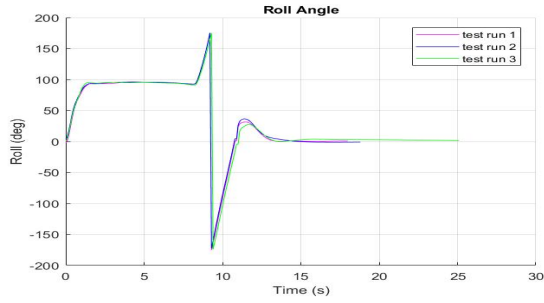


(f) Yaw rate, r , of the aircraft

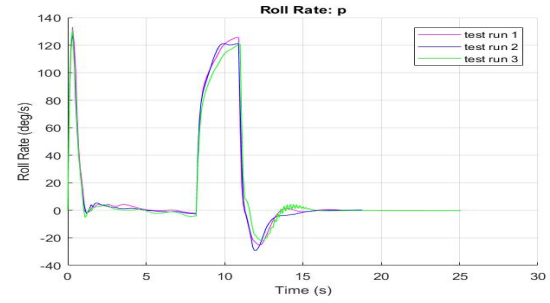


(g) Altitude of the aircraft

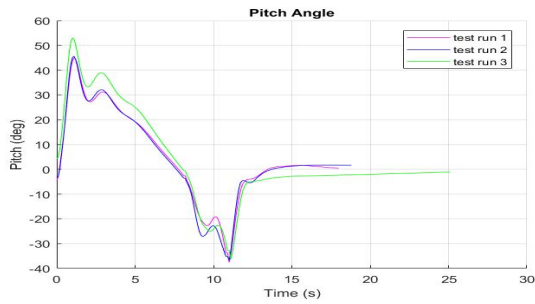
Fig. 12 States for the aircraft during the Knife Edge maneuver for test runs with varying initial conditions



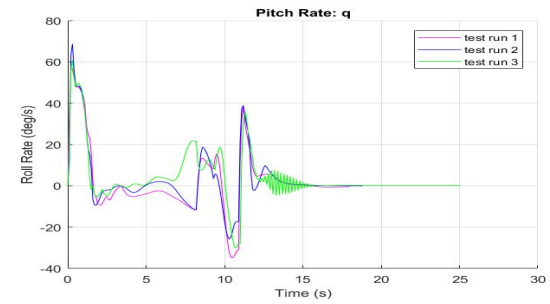
(a) Roll angle, ϕ , of the aircraft



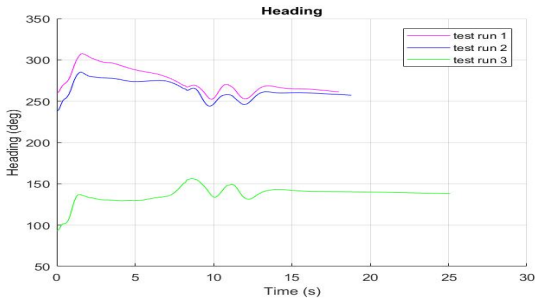
(b) Roll rate, p , of the aircraft



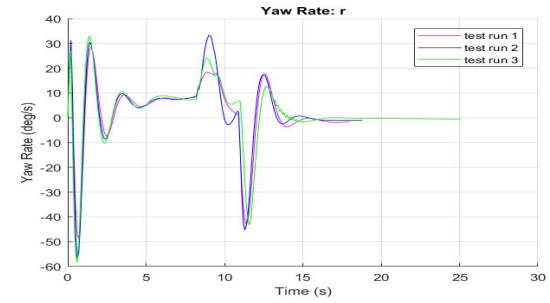
(c) Pitch angle, θ , of the aircraft



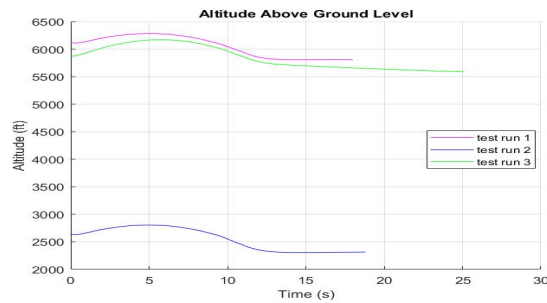
(d) Pitch rate, q , of the aircraft



(e) Heading, ψ , of the aircraft



(f) Yaw rate, r , of the aircraft



(g) Altitude of the aircraft

Fig. 13 States for the aircraft during the maneuver sequence for test runs with varying initial conditions